

rpi-digital-twin

Sistema Embarcado de Telemetria Inercial e Gêmeo Digital para Detecção Cinemática de Acidentes.

Repositório para o projeto da disciplina PCS3732 - Laboratório de Processadores.

Sobre o projeto

Protótipo IoT de telemetria inercial (IMU MPU6050 + Raspberry Pi) com Gêmeo Digital (Digital Twin) em tempo real, inspirado em sistemas como eCall e Crash Detection, para identificar padrões cinemáticos característicos de acidentes de motocicleta (impacto, queda de velocidade e inclinação anômala).

Documentação

A documentação do projeto está organizada em Markdown na pasta [docs/](#):

1. [Motivação e Justificativa](#)
2. [Objetivos](#)
3. [Requisitos do Sistema](#)
4. [Arquitetura Proposta](#)

Estrutura do repositório

```
backend/  # Raspberry Pi: leitura do MPU6050, filtro
          complementar,
          # estimativa de velocidade (ZUPT) e servidor Socket.IO
front/    # Dashboard e Gêmeo Digital 3D (React + Three.js +
          TanStack Start)
docs/     # Documentação do projeto (motivação, objetivos,
          requisitos, arquitetura)
```

Como executar

Backend (Raspberry Pi)

```
cd backend
pip install -r requirements.txt
./iniciar_wifi.sh      # sobe o Hotspot Wi-Fi "Robo-Network"
```

```
python3 main.py          # inicia o loop de telemetria (WebSocket
na porta 8765)
```

Frontend

```
cd front
npm install
npm run dev
```

Licença

Este projeto está licenciado sob os termos do arquivo [LICENSE](#).

1. Motivação e Justificativa

Os acidentes de trânsito envolvendo motociclistas representam uma das principais causas de trauma grave no transporte urbano. Devido à ausência de uma estrutura de proteção ao redor do condutor, a rapidez no socorro médico após uma colisão ou queda é o fator determinante para a sobrevivência e redução de sequelas. No entanto, em rodovias ou locais isolados, o tempo de resposta pode ser severamente prejudicado pela incapacidade da vítima de solicitar ajuda.

Projetos e sistemas consolidados na indústria automotiva — como o **eCall** (sistema europeu de chamada de emergência obrigatório em veículos) e as funcionalidades de **Deteccção de Acidentes (Crash Detection)** presentes em *smartwatches* e smartphones topo de linha — demonstram a eficiência do uso de unidades de medição inercial (IMUs) para a identificação automática de impactos e capotamentos.

O Protótipo

Este projeto propõe o desenvolvimento de um protótipo IoT compacto de telemetria inercial e **Gêmeo Digital (Digital Twin)** em tempo real. O dispositivo é fixado ao veículo/condutor para capturar dados cinemáticos continuamente. A lógica de detecção baseia-se no padrão cinemático característico de acidentes de motocicleta:

1. **Pico de aceleração ou desaceleração abrupta** (impacto frontal/lateral ou frenagem de emergência);

2. **Queda imediata da velocidade linear** (parada repentina do veículo);
3. **Movimentos angulares caóticos ou inclinação excessiva persistente (Roll/Pitch)** que indicam tombo ou rotação anômala do condutor fora da posição vertical normal.

Com o processamento desses dados na borda (*edge computing*) via Raspberry Pi e a transmissão instantânea via WebSocket para um Dashboard com modelo 3D em tempo real, o sistema demonstra a viabilidade de monitoramento cinemático contínuo para suporte à decisão e alertas de emergência.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um sistema embarcado de telemetria inercial e visualização em tempo real (Gêmeo Digital) capaz de capturar, filtrar e transmitir a cinemática de um protótipo móvel via Wi-Fi/WebSocket, permitindo o monitoramento de orientação e a identificação visual de padrões cinemáticos associados a impactos e quedas.

2.2 Objetivos Específicos

- **Integração de Hardware:** Conectar e calibrar a unidade de medição inercial (IMU MPU6050) ao barramento I2C da Raspberry Pi.
 - **Filtragem Cinemática:** Implementar algoritmo de Filtro Complementar para estimativa estável de atitude (Roll, Pitch e Yaw) e algoritmo de estimativa de velocidade com ZUPT (*Zero Velocity Update*) para atenuação de *drift*.
 - **Comunicação sem Fio Autônoma:** Configurar a Raspberry Pi como um Ponto de Acesso Wi-Fi (Hotspot) autônomo e disponibilizar um servidor Socket.IO para *broadcast* contínuo de dados na porta 8765.
 - **Gêmeo Digital e Dashboard:** Desenvolver uma aplicação Web (React + Three.js) que consuma a telemetria a 50 Hz e renderize a orientação e movimento do protótipo em 3D de forma fluida.
-

3. Requisitos do Sistema

3.1 Requisitos Funcionais (RF)

ID	Descrição
RF01	O sistema deve ler continuamente os dados brutos de aceleração (3 eixos) e velocidade angular (3 eixos) do MPU6050.
RF02	O sistema embarcado deve calibrar automaticamente os <i>offsets</i> (<i>biases</i>) dos sensores na inicialização em repouso.
RF03	O sistema deve calcular a orientação espacial (Roll, Pitch e Yaw) em graus utilizando Filtro Complementar.
RF04	O sistema deve estimar a velocidade linear aproximada do protótipo removendo o vetor de gravidade e aplicando ZUPT em momentos de repouso.
RF05	A Raspberry Pi deve atuar como servidor WebSocket/Socket.IO transmitindo o pacote JSON de telemetria na porta 8765 no evento <i>telemetry</i> .
RF06	A aplicação Web (Dashboard) deve se conectar à rede Wi-Fi gerada pelo robô/protótipo (Robo-Network) e consumir o evento de telemetria em tempo real.
RF07	A interface Web deve exibir um modelo 3D (Gêmeo Digital) cuja rotação espacial acompanhe os ângulos recebidos sem travar a interface.
RF08	A interface Web deve permitir alternar entre o modo de simulação (Mock) e conexão real com o dispositivo físico.

3.2 Requisitos Não Funcionais (RNF)

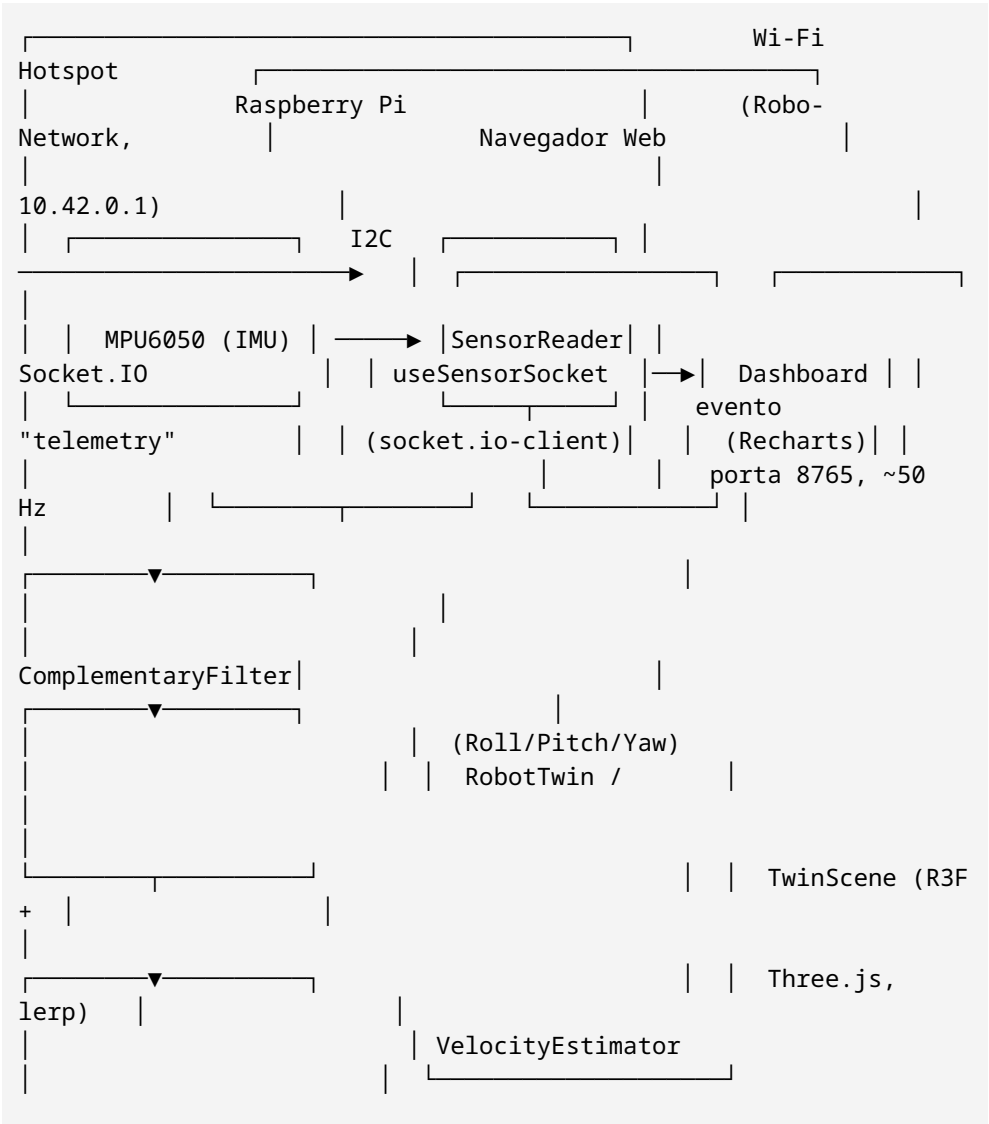
ID	Categoria	Descrição
RNF01	Frequência de Amostragem	O loop principal de amostragem e filtragem na Raspberry Pi deve rodar a uma frequência estável de 50 Hz (período de 20 ms).
RNF02	Latência	A latência média de transmissão via WebSocket no Hotspot local deve ser inferior a 100 ms para garantir fidelidade ao movimento real.
RNF03	Portabilidade / Mobilidade	O módulo embarcado (Raspberry Pi + MPU6050) deve operar alimentado por bateria (Power Bank ou regulador Buck 5V/3A) sem cabos de rede ou força.
RNF04	Confiabilidade I2C	O código Python de leitura deve tratar erros do barramento I2C, descartando quadros ruidosos sem interromper o loop.
RNF05		A renderização do modelo 3D no Three.js deve utilizar interpolação linear (<i>lerp</i>) no loop de

ID	Categoria	Descrição
	Suavidade Visual (UI)	animação, evitando re-renders custosos do React a cada pacote do Socket.
RNF06	Autonomia de Rede	O robô/protótipo deve criar sua própria rede Wi-Fi (Hotspot) com IP fixo de gateway (10.42.0.1), dispensando roteadores externos.

4. Arquitetura Proposta

4.1 Visão Geral

O sistema é dividido em dois subsistemas que se comunicam via Wi-Fi/WebSocket: um **backend embarcado** (Raspberry Pi + IMU MPU6050), responsável por amostragem, calibração e filtragem cinemática, e um **frontend Web** (React + Three.js), responsável por consumir a telemetria e renderizar o Gêmeo Digital.




```
"linear_velocity": { "x": 0.5, "y": 0.0, "z": 0.0 },  
"orientation": { "roll": 1.2, "pitch": -0.5, "yaw": 45.0 }  
}
```

4.3 Frontend / Gêmeo Digital (front/)

Aplicação Web construída com **TanStack Start (React 19)**, **Three.js** via `@react-three/fiber/@react-three/drei`, **Tailwind CSS** e **socket.io-client**:

- **services/websocket.ts** e **hooks/useSensorSocket.ts** — conectam ao servidor Socket.IO do robô e expõem os pacotes de telemetria recebidos, com suporte a modo Mock e conexão real (RF08).
- **hooks/usePositionTracker.ts** — integra a velocidade linear recebida para estimar a posição do protótipo no espaço.
- **components/TwinScene.tsx**, **RobotTwin.tsx**, **TrailPath.tsx** — cena 3D do Gêmeo Digital: aplicam a orientação (Roll/Pitch/Yaw) ao modelo com interpolação linear (*lerp*) no loop de animação do R3F, evitando re-renders do React a cada pacote do socket (RNF05, RF07).
- **routes/dashboard.tsx** — painel com gráficos (Recharts) da telemetria.
- **routes/twin.tsx** — rota do Gêmeo Digital 3D.
- **components/Sidebar.tsx** — navegação entre Home, Dashboard e Twin.

4.4 Comunicação

- Transporte: Wi-Fi local via Hotspot dedicado (sem depender de infraestrutura externa), com Socket.IO sobre WebSocket na porta 8765 (RF05/RF06/RNF02).
- Frequência: amostragem e *broadcast* a 50 Hz no backend (RNF01); o frontend consome esse fluxo e interpola visualmente para manter fluidez (RNF05).

4.5 Próximos Refinamentos

- Implementar a lógica de **detecção de acidente** (pico de aceleração + queda de velocidade + inclinação anômala) descrita na motivação, hoje ainda não presente como módulo dedicado no backend.
 - Adicionar testes automatizados de integração cobrindo o pipeline completo sensor → filtro → WebSocket → frontend.
-